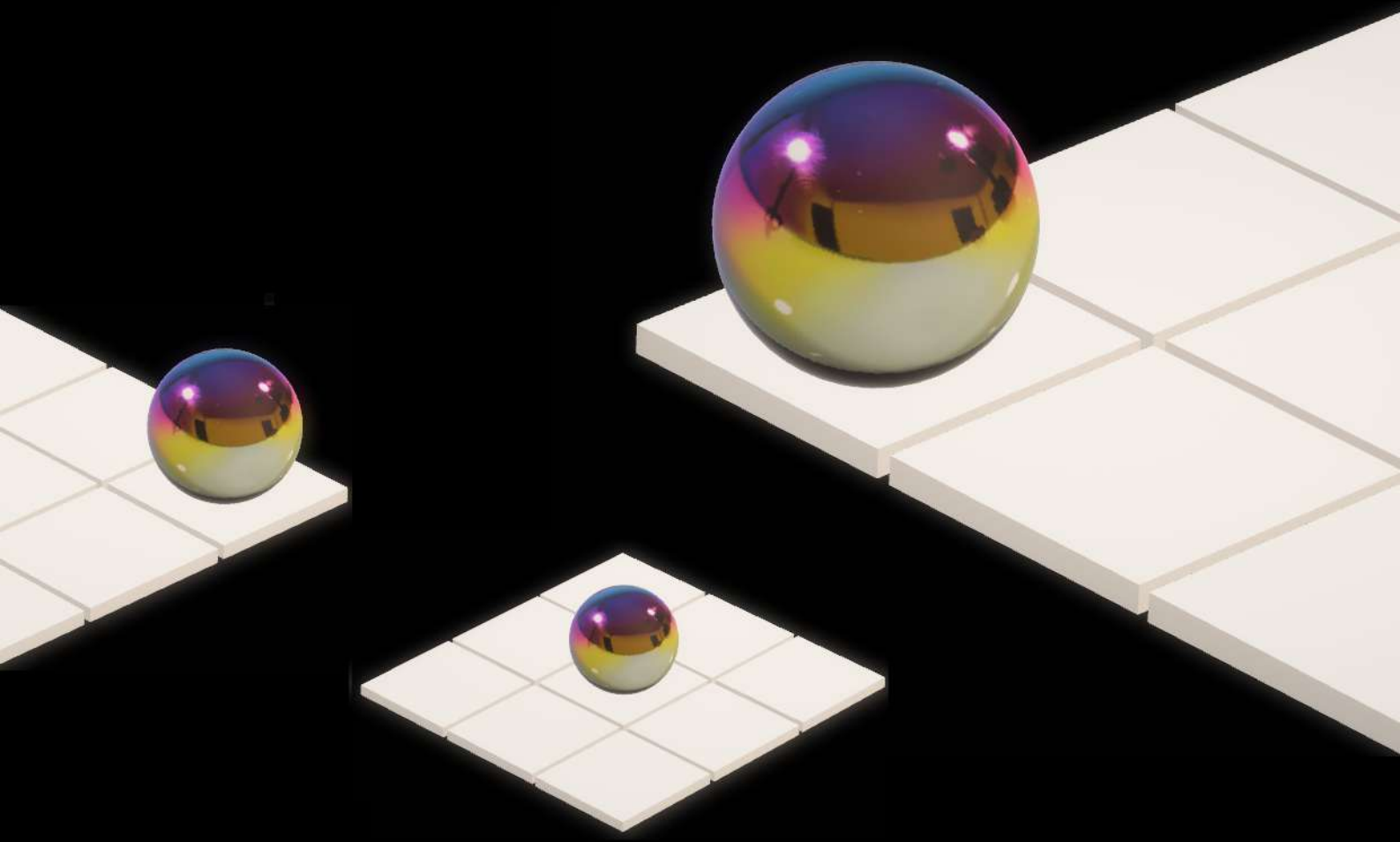
LOGIC COMPONENTS

LUKE NEO-SHELL



LUKE'S.CORP

NEO-Shell

Complete Command Reference Guide

Logic-Software-NEO-Shell® Version I — Command Introduction

Logic Components Corp · By Fogli Luca · 2024 – 2025

Table of Contents

1. Introduction and Language Structure
2. Variables — var / killvar / allvar
3. Expressions and Calculations ONE FOR ALL! — "$"
4. Input Reading — in
5. Print Output — print / wait
6. Blocks and Functions — blk / vob
7. Flow Control — if / elseif / else
8. Loops — for / while / loop / switch
9. Control Signals — break / continue / return
10. I/O Ports — out
11. EEPROM — eepset / eeplist / eeprun / eepcls / eepmem
12. I2C Protocol — iset / ifind
13. System Variables
14. Special Commands — boot / Empty
15. Complete Quick Reference
A. Appendix A — System Messages (MSGbox)

1. Introduction and Language Structure

NEO-Shell is an interpreted scripting language designed for the ATmega1284P microcontroller. It allows executing commands in real time via the serial port at 115200 baud or via I2C bus, saving persistent programs in the device EEPROM, and managing variables, loops, conditions, hardware inputs/outputs.

Basic Syntax

The semicolon (;) is a SEPARATOR between commands on the same line, NOT a mandatory terminator. A single command does not need a semicolon. Code blocks are enclosed in { }. Conditions in ( ). Block values in [ ].

```
out d 13 1                          // single command: no ; needed
out d 13 1 ; wait 500 ; out d 13 0  // multiple commands: separated
by ;
var A = 10; var B = 20; print $A    // another separation example
if ($A > 5) { print "ok\n" }       // block: ; is optional here
```

Fundamental Rules

Rule	Explanation
Separator ;	The ; separates commands on the same line. Not required for a single command.
Variables with $	To read a variable's value always use $NAME
Blocks { }	Delimit the body of if, for, while, loop, switch
Conditions ( )	Enclose the condition of if, while, for, switch
Block function [ ]	Enclose command body in a block: blk name = [function]
Case sensitivity	Variable and command names are case-sensitive
Max variables	256 simultaneous variables in memory
Max I2C string	32 bytes per single I2C message (hardware limit)

2. Variables — var / killvar / allvar

var — Variable Declaration and Assignment
Creates a new variable or updates the value of an existing one. The variable type is automatically detected by the system.

Detected Type	Condition
int	The value is an integer (e.g.: 10, -3, 0)
float	The value has a decimal part (e.g.: 3.14, -0.5)
char	The value is a single character (e.g.: A, z, !)
string	The value is any other string (e.g.: Hello world!, on)

Syntax	Description
`var NAME = value`	Creates variable with direct value
`var A = 10, B = 2.05, C = !`	Multiple assignment on same line (comma separated)
`var A = $B`	Copies the value of another variable
`var $POINTER = value`	Assigns to the name contained in the POINTER variable (indirection)

Examples

```
var Temperatura = 23   // int
var Tensione = 3.14    // float
var Lettera = A        // char
var Messaggio = CiaoMondo  // string
var X = 10, Y = 20, Z = 0  // multiple
var Doppio = $X        // copy X into Doppio
var Nome = pippo
var $Nome = 42         // creates/updates 'pippo' = 42 (indirection)
```

killvar — Removing Variables from Memory
Deletes one or more variables. Without arguments deletes all. Supports comma list and range with @.

Syntax	Description
`killvar`	Deletes ALL variables (reset)
`killvar NAME`	Deletes variable NAME
`killvar A,B,C`	Deletes A, B and C
`killvar A@Z`	Deletes all vars from A to Z in creation order

```
killvar  // total reset
killvar Temp  // delete Temp
killvar A,B,C  // delete three variables
```

allvar — Displaying Variables in Memory
Prints the variables allocated in memory. WARNING: output format depends on how it is called.

Syntax	Output	Description
`allvar`	NAME = value	Shows ALL variables in 'name = value' format
`allvar NAME`	value	Prints ONLY THE VALUE of NAME (without the name)
`allvar A,B,C`	value\nvalue\nvalue	Prints only values of A, B, C one per line
`allvar A@Z`	value\n...	Prints only values in range A-Z (by creation order)

⚠ *With parameter (name, list, range) allvar prints ONLY the value — without the name. Only allvar without arguments shows 'name = value'.*

```
allvar  // NAME = value for all
allvar X  // prints: 42 (only value of X)
allvar A@C  // prints values of A, B, C one per line
```

3. Expressions and Calculations ONE FOR ALL! — "$"

The $ allows performing a mathematical/logical expression and assigns the result to a variable. Supports operators, mathematical functions, expressions with nested parentheses, variable reading with $NAME and result type specification with a suffix.

```
$ NAME = expression;
```

Arithmetic and Bitwise Operators

Operator	Description	Example	Result
+	Addition	$A = 5 + 3	8
-	Subtraction	$A = 10 - 4	6

*	Multiplication	$A = 3 * 4	12
/	Division	$A = 10 / 4 f	2.5
%	Modulo (remainder)	$A = 10 % 3	1
@	Power (right-assoc.)	$A = 2@10	1024
&	Bitwise AND	$A = 1 & 0	0
|	Bitwise OR	$A = 1 | 0	1
^	Bitwise XOR	$A = 1 ^ 1	0
~	Bitwise NOT (unary prefix)	$A = ~0	-1
!	Logical NOT (unary prefix)	$A = !0	1
++	Increment	$A++	A = A+1
--	Decrement	$A--	A = A-1

Special Clamp / Internal Comparison Operators

WARNING: inside $ expressions the operators > < = do NOT return 0/1 as in classic languages. They have 'numeric clamp' behavior:

Operator	Actual Behavior	Example	Result
A > B	Returns B if A > B, otherwise A (high clamp)	$R = 100 > 50	50
A < B	Returns B if A < B, otherwise A (low clamp)	$R = 10 < 50	50
A = B	Returns B if A == B, otherwise A	$R = 5 = 5	5

⚠ For real boolean comparisons (true/false) use if ($A > $B) structures. The symbols > < = used OUTSIDE if/while are clamp operators, not comparison operators.

Mathematical Functions

Function	Description	Example
$var = sin N	Sine of N degrees	$R = sin90 → 1
$var = cos N	Cosine of N degrees	$R = cos0 → 1
$var = tan N	Tangent of N degrees	$R = tan45 → 1
$var = r2 N	Square root of N	$R = r2144 → 12
$var = r3 N	Cube root of N	$R = r327 → 3
$var = log N,B	Logarithm of N in base B	$R = log100,10 → 2
$var = map V,iMin,iMax,oMin,oMax	Linear remapping	$R = map512,0,1023,0,100 → 50
$var = rnd MIN,MAX	Random number (TrueRandom hardware)	$R = rnd1,100 → 1..99

- *Trigonometric functions (sin, cos, tan) receive degrees, not radians.*
- *rnd uses a TrueRandom system, it is not a pseudo-random system.*
- *map remaps linearly: map(value, inMin, inMax, outMin, outMax).*

Result Type Specifiers

Adding a letter as the last token separated by a space (or attached at the end of the number) forces the result type:

Suffix	Type	Example	Output
i (default)	int — integer part	$A = 10 / 3 i	3
f	float — decimals	$A = 10 / 3 f	3.333333
c	char — ASCII character	$A = 65c	'A'
s	string — string	$A = 3.14 s	"3.14"

Complete Examples

```
$A = 10 + 5     // A = 15 (int)
$B = $A * 2     // B = 30
$C = r2144      // C = 12 (square root of 144)
$D = (10 + 20) / 3 f  // D = 10.0 (float)
$E = 2@8        // E = 256 (2 to the power of 8)
$F = sin90      // F = 1
$G = rnd1,100   // G = random number 1-99
$H = map512,0,1023,0,255  // H = ADC → PWM remapping
$I = log1000,10  // I = 3
$J = !0         // J = 1 (logical NOT)
$K = ~255       // K = -256 (bitwise NOT)
$A++            // increment A
$A--            // decrement A
```

- *A single alphabetic character used as an operand in $ expressions is automatically converted to its ASCII code. E.g.: $X = A → X = 65.*

4. Input Reading — in

The in command reads a value from hardware or from the user and assigns it to a variable. It is always used inside a $ expression:

```
$VARNAME = in TYPE SOURCE PIN;
```

Type	Source	Description
$var = in D PIN	Pin / hardware	Reads digital pin (0 or 1). Sets the pin as INPUT.
$var = in A PIN	Pin / hardware	Reads analog pin (0-1023, 10 bit).
$var = in K	default	Reads ONE single character from Serial OR I2C (whichever first)
$var = in KS	Serial	Reads a character only from Serial port
$var = in KI	I2C	Reads a character only from I2C

$var = in T	default	Reads a line of text (up to \n) from Serial OR I2C
$var = in TS	Serial	Reads a line of text only from Serial port
$var = in TI	I2C	Reads a line of text only from I2C

in K and in T are blocking: execution stops until input arrives. in K has 5s timeout, in T has 10s timeout.
Before in K and in T the serial buffer is automatically flushed to avoid reading residual characters.

Examples

```
$Stato = in D 13  // read digital pin 13
$Pot = in A 0     // read analog pin A0 (0-1023)
$Tasto = in K     // wait for a key from serial or I2C
$TastoSerial = in KS  // wait only from serial
$Riga = in T      // wait for text line (until Enter)
$RigaI2C = in TI  // wait for line only from I2C

// Example: interactive menu
print "Press 1 for LED on, 2 for LED off:\n"
$Scelta = in KS
if ($Scelta == 1) { out d 13 on; }
if ($Scelta == 2) { out d 13 off; }

// Example: analog reading with remapping
$Valore = in A 0
$PWMval = map $Valore,0,1023,0,255
out a 9 $PWMval  // apply value to hardware PWM
```

5. Print Output — print / wait

print — Print Text and Values to Serial or I2C
Prints to serial or to an I2C device. Distinguishes two modes based on the presence of quotation marks:

Context	Behavior
"X" (inside quotes)	Literal text. $NAME is expanded with the variable value. \n \t \r sequences active.
X (outside quotes)	Evaluated as mathematical expression via $NEO. Variables, operators, functions.
"X" Y "Z" (mixed)	Parts inside and outside quotes are concatenated into a single output.
I2C @@ "X"	print @ADDRESS@ content — two @ required. 32 byte limit.

Valid Escape Sequences Inside Quotes

Sequence	Character Produced
\n	New line (LF, 0x0A)
\t	Tab (TAB, 0x09)

\r	Carriage return (CR, 0x0D)
\X	Any other character X is printed literally (e.g.: \" prints a quote)

⚠ I2C syntax is: print @ADDRESS@ "text" with TWO @ symbols: before and after the address.

Examples

```
print "Welcome!\n"
var T = 23
print "Temperature: $T degrees\n"
$S = 10 + 20
print "Sum = " $S "\n"  // mixed: text + expression
print $T * 2  // outside quotes: prints 46
print @0x08@ "out d 13 on"  // send via I2C
```

wait — Execution Pause (milliseconds)
Suspends execution for the specified number of milliseconds. The parameter can be a direct value or a variable with $.

```
wait 1000  // pause 1 second
wait 500   // pause 500 ms
var T = 2000
wait $T    // pause 2 seconds from variable
```

6. Blocks and Functions — blk / vob

NEO-Shell allows defining reusable code blocks similar to functions. A block is defined with blk and saved in a variable. It can accept parameters through vob placeholders declared inside it.

blk — Defines a Code Block (Function)
Creates a variable whose value is an entire code block enclosed in [ ]. The block is called by simply writing its name as a command. If it contains vob, it accepts parameters at call time.

Syntax	Description
`blk NAME = [ ... ]`	Creates block with direct name
`blk $NAMEVAR = [ ... ]`	Creates block with name taken from NAMEVAR value (indirection)

```
blk Saluta = [ print "Hello from block!\n" ]
Saluta  // calls the block
```

vob — Formal Parameter of a Block (Variable Of Block)
Declared inside a blk to indicate it accepts a parameter. The optional separator (in double quotes after the name) specifies the character/string separating the next parameter. Default: space.

Syntax	Description
`vob NAME`	Parameter separated by space (default)
`vob NAME ","`	Parameter separated by comma

`vob NAME "7"`	Parameter separated by a number
`vob NAME "%"`	Parameter separated by percentage

⚠ If using ";", there is a risk that the interpreter may take the next value as a command to execute instead of a value to associate with the next vob.
The last vob of a block always collects all remaining characters, regardless of the separator.

Example: block with two parameters (space separator)

```
blk Somma = [
  vob A;
  vob B;
  $R = $A + $B;
  print "Result: " $R "\n"
]
Somma 10 20  // A=10, B=20 → Result: 30
```

Example: custom separator

```
blk Info = [
  vob Nome ","  // first param up to the comma
  vob Eta       // second param: rest of string
  print "Nome: " $Nome " Eta: " $Eta "\n"
]
Info Mario,25  // Nome=Mario, Eta=25
```

7. Flow Control — if / elseif / else

NEO-Shell supports complete conditional structures. Conditions use comparison and logical operators. Comparisons work on both numbers (numeric comparison with floating point tolerance) and strings (lexicographic comparison).

Operator	Meaning	Notes
==	Equal	Numeric (tolerance 0.0001) or lexicographic
!=	Not equal	Numeric or lexicographic
>	Greater than	
<	Less than	
>=	Greater or equal	
<=	Less or equal	
&&	Logical AND	Combines multiple conditions
||	Logical OR	Combines multiple conditions

if — Conditional Execution

```
if (condition) { instructions }

var X = 10
if ($X > 5) { print "X > 5\n" }
```

```
if ($X >= 10 && $X < 20) { print "between 10 and 19" }
if ($X == 10 || $X == 20) { print "10 or 20" }
```

elseif — Alternative Conditional Branch
Must immediately follow an if (or another elseif). Executed only if all previous branches were false.

```
elseif (condition) { instructions }
```

else — Default Branch (otherwise)

```
else { instructions }
```

Complete Example — if/elseif/else chain

```
var Voto = 75
if ( $Voto >= 90 ) { print "Excellent!" }
elseif ( $Voto >= 70 ) { print "Good" }
elseif ( $Voto >= 60 ) { print "Pass" }
else { print "Fail" }
```

8. Loops — for / while / loop / switch

for — Loop with Initialization, Condition and Increment
Classic loop with three parts separated by ; in the condition. The variable is automatically declared with var. Supports break, continue, return.

```
For ( i = 0; $i < 10; $i++) { instructions }
```

> ⚠ *The for uses simple calculation expressions: logical operators && and || do NOT work in the for condition. For complex conditions use an internal if with break.*

```
// Count from 0 to 9
for ( i = 0; $i < 10; $i++) { print $i "\n" }

// Loop with decrement
for ( k = 10; $k > 0; $k--) {
  out d 13 on
  wait 100
  out d 13 off
  wait 100
}
```

while — Loop with Entry Condition
Repeats the block while the condition is true. The condition is evaluated before each iteration.

```
while (condition) { instructions }

var N = 0
while ( $N < 5 ) { print $N "\n" ; $N++ }
```

loop — Infinite Loop
Executes the block indefinitely without a condition. Use break or return to exit. Ideal for continuous monitoring programs.

```
loop { instructions }
```

```
var C = 0
loop { $C++ ; print "Tick " $C "\n" ; wait 100 ; if ($C >= 60)
{ break } }
// exits after 60 iterations
```

switch — Multiple Value Selection
Compares the value of a variable with a series of cases. If no case matches, executes the DEFAULT block (if present). Each case normally ends with break to avoid executing subsequent cases (fall-through).

```
switch ($VARIABLE) {
  case VALUE1 { instructions; break }
  case VALUE2 { instructions; break }
  DEFAULT { instructions }
}
```

Complete Example

```
var Giorno = 3
switch ($Giorno) {
  case 1 { print "Monday" ; break }
  case 2 { print "Tuesday" ; break }
  case 3 { print "Wednesday" ; break }
  case 4 { print "Thursday" ; break }
  DEFAULT { print "Weekend" }
}
```

9. Control Signals — break / continue / return

break — Exit Current Loop or Switch
Immediately interrupts the execution of the loop (for, while, loop) or switch it is in. Execution resumes from the instruction after the block.

```
var I = 0
while (1 == 1) { $I++ ; if ($I >= 5){ break } }
print "Exited with I = " $I "\n"  // prints: 5
```

continue — Skip to Next Loop Iteration
Interrupts the current iteration and moves to the next one. In a for, executes the increment before returning to the condition.

```
for ( k = 0; $k < 10; $k++) { if ($k == 5) { continue }; print $k " "
}
// skip when k=5
// Output: 0 1 2 3 4 6 7 8 9
```

return — Exit Current blk Block (with optional value)
Terminates execution of the current blk block and returns control to the caller. Can optionally return a value saved in the $return system variable.

Syntax	Description
`return;`	Exits the blk without return value

`return value;`	Exits the blk and saves 'value' in $return
`return $EXPR;`	Exits the blk and saves the value of $EXPR in $return

⚠ return is scoped to the blk: the shouldReturn flag is automatically reset after the block execution ends. A return inside a blk does NOT propagate outside — the calling code continues normally.

```
blk Massimo = [ vob A ; vob B ; if ($A > $B) { return $A }; return $B
]
Massimo 10 20
print "Max = " $return "\n"  // prints: Max = 20
```

10. I/O Ports — out

The out command manages all hardware outputs. The output type is the letter immediately after out (d, a, t, s, p). Parameter values can be constants or $ variables.

Command	Description	Example
`out d PIN on`	Digital output HIGH	out d 13 on
`out d PIN off`	Digital output LOW	out d 13 off
`out d PIN hi`	Digital output HIGH (alias)	out d 13 hi
`out d PIN lo`	Digital output LOW (alias)	out d 13 lo
`out d PIN 1`	Digital output HIGH (numeric)	out d 13 1
`out a PIN val`	Hardware PWM analogWrite (0-255)	out a 9 128
`out t PIN freq`	Tone: continuous sound on PIN	out t 8 440
`out t PIN freq dur`	Tone with duration in milliseconds	out t 8 440 500
`out t PIN 0`	Deactivate tone (noTone)	out t 8 0
`out s PIN degrees`	Servo: position at N degrees (0-180)	out s 6 90
`out p PIN freq duty`	Software PWM: freq Hz, duty %	out p 5 1000 50
`out p PIN freq duty dur`	Software PWM with duration ms	out p 5 1000 50 3000
`out p PIN 0 0`	Stop software PWM on PIN	out p 5 0 0

□ out p generates software PWM: supports up to 5 simultaneous signals on any pin. Frequency and duty cycle are completely free. With freq=0 or duty=0 the signal is stopped.

Using Variables in Parameters

```
var PIN = 13 , Stato = 110
out d $PIN on   // use PIN from variable
out a 9 $Stato  // use Stato as PWM value
```

Practical Examples

```
// Blink LED
```

```
loop { out d 13 on ; wait 500 ; out d 13 off ; wait 500 }

// Servo sweep
for ( pos = 0 ; $pos <= 180 ; $pos++ ) { out s 13 $pos; wait 15 }

// Software PWM 2kHz, 30% duty, 5 seconds
out p 5 2000 30 5000;
```

11. EEPROM — eepset / eeplist / eeprun / eepcls / eepmem

EEPROM allows saving a NEO-Shell program that survives power-off. Lines are numbered from 1. Line 1 is reserved for the boot marker. The program is executed with eeprun.

eepset — Writing, Editing and Manipulating EEPROM Lines

Syntax	Description
`eepset`	Interactive editor mode — type lines, end with 'end'
`eepset N text`	Writes/replaces 'text' in line N
`eepset +N text`	Inserts 'text' as new line at position N (shifts subsequent lines)
`eepset +N`	Inserts an empty line at position N
`eepset => A B`	Swaps content of line A with line B
`eepset => A@B C@D`	Swaps line group A-B with group C-D
`eepset <= A B`	Same as =>, but in reverse direction (B → A)
`$ VAR = eepset N`	Reads line N from EEPROM and saves it in VAR

eepset automatically substitutes $NAME variables in the text to write. To write a literal $ without expansion use $$NAME — the double $$ is reduced to a single $.
The reading $ VAR = eepset N uses the $ command syntax (not var). Example: $ Line = eepset 3;

```
eepset  // start interactive editor
```

The interactive editor shows real-time feedback for each inserted line:

```
TEXT << Used: 120 Free: 903 bytes >>
```

To exit the editor type 'end'. The editor accepts input from both Serial port and I2C.

This way, you can also modify individual strings of the code by first entering the number of the line to edit and then entering the new text or command.

```
eepset 1 boot           // line 1 = boot marker
eepset 2 var LED = 13   // write code in line 2
eepset +3 print "inserted"  // insert new line at pos. 3
eepset +5               // insert empty line at pos. 5
eepset => 2 5           // swap lines 2 and 5
eepset <= 2 5           // swap in reverse direction
eepset 2 print $$Var    // writes literally: print $Var
$ Riga = eepset 2       // read line 2 into variable Riga
```

eeplist — Displaying EEPROM Contents

Syntax	Description
`eeplist`	Shows all lines with number and content

`eeplist N`	Shows only line N
`eeplist N@M`	Shows lines from N to M inclusive
`eeplist N,M,K`	Shows lines N, M and K (arbitrary list)

```
eeplist         // show all
eeplist 1@5     // show lines 1-5
eeplist 2,4,6   // show lines 2, 4, 6
```

eeprun — Executing EEPROM Program

Syntax	Description
`eeprun`	Executes ALL lines (line 1 'boot' is skipped)
`eeprun N`	Executes only line N
`eeprun N>`	Executes from line N to the end
`eeprun N@M`	Executes lines from N to M inclusive
`eeprun N,M,K`	Executes lines N, M and K in that order

```
eeprun         // run program
eeprun 5       // run only line 5
eeprun 2@8     // run lines 2-8
eeprun 3>      // run from line 3 onwards
eeprun 2,5,7   // run lines 2, 5 and 7
```

eepcls — Clearing EEPROM Lines

Clears lines from EEPROM. There are TWO clearing modes:

Mode	Syntax	Behavior
Clear (default)	eepcls N	Replaces line with 'Empty' — line remains but is empty
Physical delete	eepcls N!	Completely deletes line — subsequent lines compact
Range clear	eepcls N@M	Clears lines from N to M
Range delete	eepcls N!@M!	Physically deletes lines from N to M
Mixed list	eepcls N,M!,K	N cleared, M deleted, K cleared
Complete format	eepcls	Clears ALL EEPROM (byte by byte)

⚠ eepcls without ! clears the line (writes 'Empty'). With ! physically deletes it, compacting subsequent lines. Line numbers change after a physical deletion.

```
eepcls        // total EEPROM format
eepcls 5      // line 5 → 'Empty'
eepcls 5!     // line 5 physically deleted
eepcls 2@4    // lines 2,3,4 → 'Empty'
eepcls 2!@4!  // lines 2,3,4 deleted
```

eepmem — EEPROM Memory Statistics

Shows used bytes and remaining free bytes in EEPROM.

```
eepmem  // e.g.: MEM Bytes used 120 / MEM Bytes free 903
```

12. I2C Protocol — iset / ifind

NEO-Shell operates simultaneously as I2C MASTER and SLAVE. As slave it responds to the default address 0x12. As master it can send commands to other devices. The limit for a single message is 32 bytes.

iset — Sets the I2C Target Address for Transmissions
Changes the I2C destination address used by print @...@. Accepts address in hexadecimal (0xNN) or decimal format. Reinitializes the Wire bus with the new slave address.

Syntax	Description
`iset 0x08`	Sets target to 0x08 (NEO default)
`iset 0x12`	Sets target to 0x12
`iset $VAR`	Uses the variable value as address

```
iset 0x08  // change target to 0x08
```

- *Valid addresses: 0x08 – 0x77. Outside this range 'Addr Invalid' error is emitted.*

ifind — I2C Bus Scan
Without parameters performs a complete scan (0x08–0x77) and prints all found devices. With an address parameter checks only that device.

Syntax	Description
`ifind`	Complete scan — prints all found addresses
`ifind 0x08`	Checks if address 0x08 exists
`ifind $VAR`	Checks the address contained in $VAR

When an I2C address is searched, NEO-Shell sets the "$I2C" variable. If the address exists and is connected, the value will be 1 (TRUE), otherwise it will be 0 (FALSE).

```
ifind       // complete scan
ifind 0x08  // check 0x08
if ($I2C == 1) { print "Device found!" }
```

13. System Variables

NEO-Shell manages some special variables that are automatically updated by the system during execution.

Variable	Type	Description
$return	any	Contains the value returned by the last blk with 'return VALUE'
$I2C	int (0/1)	Set by ifind: 1 if device is found, 0 otherwise
$NEOin	string	Contains the text of the PREVIOUS command (updated before executing the new one)
$NEO	calculated	Internal temporary variable used by print to evaluate expressions outside quotes

$NEOin is updated before executing the current command: it always contains the immediately preceding command, not the one just launched.

Examples

```
// Using $return
blk Rad = [ vob X; return r2$X ]
Rad 144
print "Square root = " $return "\n"  // prints: 12

// Using $I2C
ifind 0x08
if ($I2C == 1) {
  print @0x08@ "you are connected to the NEO-Shell terminal!"
}

// Using $NEOin
print "Last command: " $NEOin "\n"
```

14. Special Commands — boot / Empty

boot — EEPROM Automatic Start Marker (line 1)
Not an executable command, but a special marker that must be in line 1 of EEPROM. At power-on, NEO-Shell checks line 1: if it contains boot, it automatically executes the subsequent lines.

Line 1 Syntax	Behavior at Startup
`boot`	Executes all lines from 2 onwards (equivalent to eeprun)
`boot N`	Executes from line N onwards
`boot N@M`	Executes lines from N to M
`boot N,M,K`	Executes lines N, M and K

The 'boot' comparison in line 1 is case-insensitive: BOOT, Boot, boot all work.

```
// Blinking LED program at startup:
eepset 1 boot
eepset 2 var LED = 13
eepset 3 loop { out d $LED on ; wait 500 ; out d $LED off ; wait
500 }
```

Empty — Empty Line Marker in EEPROM
Not an executable command. Written automatically by the interpreter when a line is cleared with eepcls without the ! modifier (logical deletion). The interpreter silently ignores all lines marked Empty during eeprun.

```
eepcls 2   // line 2 becomes 'Empty'
eeplist 2  // shows: 2 > Empty
eeprun     // line 2 is automatically skipped
```

Appendix A — System Messages (MSGbox)

NEO-Shell uses an internal library of numbered messages to communicate states, errors and confirmations. These appear automatically on the serial monitor during use. Complete table:

N°	Serial Message	When It Appears
0	Logic Components(C) / NEO-Shell Version I / Copyright...	At device startup (banner)
1	>>	Interactive prompt (ready for next command)
2	Run	Start of eeprun execution
3	Stop	End / stop execution
4	Error	Generic format or syntax error
5	Ok	Operation successful (generic)
6	Wait	System waiting
7	!	Generic warning
8	?	Input request or uncertainty
9	Cleared	Variable memory or EEPROM cleared
10	Saved	Data saved correctly (e.g.: after eepset)
11	Format Type?	Unrecognized format in a blk
12	Typo	Typing error / unrecognized command
13	Mem Empty!	Variable memory empty
14	Mem Tall!	Variable memory full (256 variables reached)
15	###	EEPROM format progress bar
16	EEProm Empty!	EEPROM empty (no lines)
17	EEProm Tall!	EEPROM full
18	MEM Bytes used	Prefix for used byte count (eepmem)
19	MEM Bytes free	Prefix for free byte count (eepmem)
20	Not Exist!	Variable or EEPROM line not found
21	End!	End of boot or eeprun execution
22	ok	I2C transmission successful (i2cSendData)
23	Error: NACK Addr	I2C: recipient did not acknowledge address
24	Error: NACK Data	I2C: recipient did not accept data
25	Error: Timeout	I2C: transmission timeout
26	Error: Data Long	I2C: data too long (>32 bytes)
27	Addr: 0x	I2C address display prefix
28	No devices	ifind: no I2C device found
29	Scanning...	ifind: bus scan in progress

30	0x	Prefix of address found during ifind
31	Addr Invalid	I2C address out of range (valid: 0x08-0x77)

15. Complete Quick Reference

Command / Syntax	Usage	Category
`var X = value`	Declare / update variable	Variables
`var A=1, B=2, C=3`	Multiple declaration	Variables
`killvar [X / A,B / A@Z]`	Delete variable(s)	Variables
`allvar [X / A,B / A@Z]`	Display variable(s)	Variables
`$ X = expr`	Math calculation, result in X	Expressions
`$ X++ / $ X--`	Increment / decrement	Expressions
`$ X = A > B (clamp)`	Clamp: returns B if A>B, else A	Expressions
`$ X = A < B (clamp)`	Clamp: returns B if A<B, else A	Expressions
`$ X = rndMIN,MAX`	TrueRandom hardware random number	Expressions
`$ X = r2N / r3N`	Square root / cube root	Expressions
`$ X = sinN / cosN / tanN`	Trig in degrees	Expressions
`$ X = logN,B`	Logarithm in base B	Expressions
`$ X = mapV,i0,i1,o0,o1`	Linear remapping	Expressions
`$ X = A@B`	Power: A raised to B	Expressions
`$ X = !V / ~V`	Logical NOT / bitwise NOT	Expressions
`$ X = inDPIN`	Read digital pin (0/1)	Input
`$ X = inAPIN`	Read analog pin 0-1023	Input
`$ X = inK[S/I]`	Read a character (S=Serial, I=I2C)	Input
`$ X = inT[S/I]`	Read a text line (S=Serial, I=I2C)	Input
`print "text"`	Print to serial	Output
`print @0xNN@ "text"`	Send via I2C (two @ required)	Output
`wait ms`	Pause in milliseconds	Output
`blk N = [ ... ]`	Define block/function N	Blocks
`vob NAME ["sep"]`	Formal parameter in a blk	Blocks
`N params`	Call block N with parameters	Blocks
`if (cond) {...}`	Conditional execution	Flow
`elseif (cond) {...}`	Alternative condition	Flow
`else {...}`	Else branch	Flow
`for (init;c;inc) {}`	Classic for loop	Loops

`while (cond) {}`	While loop	Loops
`loop {}`	Infinite loop	Loops
`switch ($X) { case ... }`	Multiple selection	Loops
`break`	Exit loop / switch	Control
`continue`	Next loop iteration	Control
`return [value]`	Exit blk, optional value in $return	Control
`out d PIN on/off/hi/lo`	Digital output	I/O
`out a PIN val`	Hardware PWM analogWrite 0-255	I/O
`out t PIN freq [dur]`	Tone (sound on pin)	I/O
`out s PIN degrees`	Servo motor 0-180	I/O
`out p PIN f d [dur]`	Software multi-channel PWM (max 5)	I/O
`eepset N text`	Write EEPROM line	EEPROM
`eepset +N [text]`	Insert line (optional text)	EEPROM
`eepset => A B`	Swap lines A and B	EEPROM
`eepset <= A B`	Swap in opposite direction	EEPROM
`$ X = eepset N`	Read line N into variable X	EEPROM
`$$NAME in eepset`	Write $NAME literally (dollar escape)	EEPROM
`eeplist [N / N@M / N,M]`	Display EEPROM lines	EEPROM
`eeprun [N / N> / N@M]`	Execute EEPROM lines	EEPROM
`eepcls [N / N! / N@M]`	Clear (default) or physically delete (with !)	EEPROM
`eepmem`	EEPROM byte statistics	EEPROM
`iset 0xNN`	Set I2C target address (0x08-0x77)	I2C
`ifind [0xNN]`	Bus scan / verify → $I2C	I2C
`$return`	Value from last return in a blk	System
`$I2C`	1=found / 0=not found (updated by ifind)	System
`$NEOin`	Text of the PREVIOUS command	System
`$NEO`	Internal temporary var (used by print)	System
`boot [N / N@M / N,M,K]`	EEPROM auto-start (line 1)	Special
`Empty`	Logically deleted line marker	Special